

PCA#

<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>

Description:

Principal component analysis (PCA). Linear dimensionality reduction using Singular Value Decomposition of the data to project it to a lower dimensional space. The input data is centered but not scaled for each feature before applying the SVD. It uses the LAPACK implementation of the full SVD or a randomized truncated SVD by the method of Halko et al. 2009, depending on the shape of the input data and the number of components to extract. With sparse inputs, the ARPACK implementation of the truncated SVD can be used (i.e. through `scipy.sparse.linalg.svds`). Alternatively, one may consider `TruncatedSVD` where the data are not centered.

Function Signature

```
class sklearn.decomposition.PCA(n_components=None, *,
copy=True, whiten=False, svd_solver='auto', tol=0.0,
iterated_power='auto', n_oversamples=10,
power_iteration_normalizer='auto', random_state=None)
[source]#
```

Parameters:

`n_components`int, float or 'mle', default=None

`copy`bool, default=True

`whiten`bool, default=False

```
class sklearn.decomposition.PCA(n_components=None, *,
copy=True, whiten=False, svd_solver='auto', tol=0.0,
iterated_power='auto', n_oversamples=10,
power_iteration_normalizer='auto', random_state=None)
[source]#
```

Parameters:

`n_components`int, float or 'mle', default=None

`copy`bool, default=True

`whiten`bool, default=False

`fit(X, y=None)`[source]#

Parameters:

`X`{array-like, sparse matrix} of shape (n_samples, n_features)

`y`Ignored

Returns:

Parameters

`X{array}`

Training data, where `n_samples` is the number of samples and `n_features` is the number of features.

`X{array}`

Training data, where `n_samples` is the number of samples and `n_features` is the number of features.

`input_featuresarray`

Only used to validate feature names with the names seen in fit.

`paramsdict`

Parameter names mapped to their values.

`Xarray`

New data, where `n_samples` is the number of samples and `n_components` is the number of components.

`X_originalarray`

Original data, where `n_samples` is the number of samples and `n_features` is the number of features.

`Xarray`

The data.

`Xarray`

The data.

`**paramsdict`

Estimator parameters.

`X{array}`

New data, where `n_samples` is the number of samples and `n_features` is the number of features.

X_newarray

Projection of X in the first principal components, where n_samples is the number of samples and n_components is the number of the components.

Code Examples

```
n_components == min(n_samples, n_features)
```

```
n_components == min(n_samples, n_features) - 1
```

```
>>> import numpy as np
>>> from sklearn.decomposition import PCA
>>> X = np.array([[ -1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1],
[3, 2]])
>>> pca = PCA(n_components=2)
>>> pca.fit(X)
PCA(n_components=2)
>>> print(pca.explained_variance_ratio_)
[0.9924 0.0075]
>>> print(pca.singular_values_)
[6.30061 0.54980]
```

```
>>> pca = PCA(n_components=2, svd_solver='full')
>>> pca.fit(X)
PCA(n_components=2, svd_solver='full')
>>> print(pca.explained_variance_ratio_)
[0.9924 0.00755]
>>> print(pca.singular_values_)
[6.30061 0.54980]
```

```
>>> pca = PCA(n_components=1, svd_solver='arpack')
>>> pca.fit(X)
PCA(n_components=1, svd_solver='arpack')
>>> print(pca.explained_variance_ratio_)
[0.99244]
>>> print(pca.singular_values_)
[6.30061]
```

Notes & Warnings

NOTE See also [KernelPCA](#), [Kernel Principal Component Analysis](#), [SparsePCA](#), [Sparse Principal Component Analysis](#), [TruncatedSVD](#), [Dimensionality reduction using truncated SVD](#), [IncrementalPCA](#), [Incremental Principal Component Analysis](#).

Detailed Documentation

Documentation

Principal component analysis (PCA).

Linear dimensionality reduction using Singular Value Decomposition of the data to project it to a lower dimensional space. The input data is centered but not scaled for each feature before applying the SVD.

It uses the LAPACK implementation of the full SVD or a randomized truncated SVD by the method of Halko et al. 2009, depending on the shape of the input data and the number of components to extract.

With sparse inputs, the ARPACK implementation of the truncated SVD can be used (i.e. through `scipy.sparse.linalg.svds`). Alternatively, one may consider `TruncatedSVD` where the data are not centered.

Notice that this class only supports sparse inputs for some solvers such as “`arpack`” and “`covariance_eigh`”. See `TruncatedSVD` for an alternative with sparse data.

For a usage example, see [Principal Component Analysis \(PCA\) on Iris Dataset](#)

Read more in the [User Guide](#).

Number of components to keep. if `n_components` is not set all components are kept:

If `n_components == 'mle'` and `svd_solver == 'full'`, Minka’s MLE is used to guess the dimension. Use of `n_components == 'mle'` will interpret `svd_solver == 'auto'` as `svd_solver == 'full'`.

If $0 < n_components < 1$ and `svd_solver == 'full'`, select the number of components such that the amount of variance that needs to be explained is greater than the percentage specified by `n_components`.

If `svd_solver == 'arpack'`, the number of components must be strictly less than the minimum of `n_features` and `n_samples`.

Hence, the `None` case results in:

If `False`, data passed to `fit` are overwritten and running `fit(X).transform(X)` will not yield the expected results, use `fit_transform(X)` instead.

When `True` (`False` by default) the `components_` vectors are multiplied by the square root of `n_samples` and then divided by the singular values to ensure uncorrelated outputs with unit component-wise variances.

Whitening will remove some information from the transformed signal (the relative variance scales of the components) but can sometime improve the predictive accuracy of the downstream estimators by making their data respect some hard-wired assumptions.

The solver is selected by a default 'auto' policy is based on `X.shape` and `n_components`: if the input data has fewer than 1000 features and more than 10 times as many samples, then the "covariance_eigh" solver is used. Otherwise, if the input data is larger than 500x500 and the number of components to extract is lower than 80% of the smallest dimension of the data, then the more efficient "randomized" method is selected. Otherwise the exact "full" SVD is computed and optionally truncated afterwards.

Run exact full SVD calling the standard LAPACK solver via `scipy.linalg.svd` and select the components by postprocessing

Precompute the covariance matrix (on centered data), run a classical eigenvalue decomposition on the covariance matrix typically using LAPACK and select the components by postprocessing. This solver is very efficient for `n_samples >> n_features` and small `n_features`. It is, however, not tractable otherwise for large `n_features` (large memory footprint required to materialize the covariance matrix). Also note that compared

to the “full” solver, this solver effectively doubles the condition number and is therefore less numerical stable (e.g. on input data with a large range of singular values).

Run SVD truncated to `n_components` calling ARPACK solver via `scipy.sparse.linalg.svds`. It requires strictly $0 < n_components < \min(X.shape)$

Run randomized SVD by the method of Halko et al.

Added in version 0.18.0.

Changed in version 1.5: Added the ‘`covariance_eigh`’ solver.

Tolerance for singular values computed by `svd_solver == ‘arpack’`. Must be of range `[0.0, infinity)`.

Added in version 0.18.0.

Number of iterations for the power method computed by `svd_solver == ‘randomized’`. Must be of range `[0, infinity)`.

Added in version 0.18.0.

This parameter is only relevant when `svd_solver="randomized"`. It corresponds to the additional number of random vectors to sample the range of `X` so as to ensure proper conditioning. See `randomized_svd` for more details.

Added in version 1.1.

Power iteration normalizer for randomized SVD solver. Not used by ARPACK. See `randomized_svd` for more details.

Added in version 1.1.

Used when the 'arpack' or 'randomized' solvers are used. Pass an int for reproducible results across multiple function calls. See Glossary.

Added in version 0.18.0.

Principal axes in feature space, representing the directions of maximum variance in the data. Equivalently, the right singular vectors of the centered input data, parallel to its eigenvectors. The components are sorted by decreasing explained_variance_.

The amount of variance explained by each of the selected components. The variance estimation uses $n_samples - 1$ degrees of freedom.

Equal to $n_components$ largest eigenvalues of the covariance matrix of X .

Added in version 0.18.

Percentage of variance explained by each of the selected components.

If $n_components$ is not set then all components are stored and the sum of the ratios is equal to 1.0.

The singular values corresponding to each of the selected components. The singular values are equal to the 2-norms of the $n_components$ variables in the lower-dimensional space.

Added in version 0.19.

Per-feature empirical mean, estimated from the training set.

Equal to $X.mean(axis=0)$.

The estimated number of components. When $n_components$ is set to 'mle' or a number between 0 and 1 (with `svd_solver == 'full'`) this number is estimated from input data.

Otherwise it equals the parameter `n_components`, or the lesser value of `n_features` and `n_samples` if `n_components` is `None`.

Number of samples in the training data.

The estimated noise covariance following the Probabilistic PCA model from Tipping and Bishop 1999. See “Pattern Recognition and Machine Learning” by C. Bishop, 12.2.1 p. 574 or <http://www.miketipping.com/papers/met-mppca.pdf>. It is required to compute the estimated data covariance and score samples.

Equal to the average of $(\min(n_features, n_samples) - n_components)$ smallest eigenvalues of the covariance matrix of X .

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when X has feature names that are all strings.

Added in version 1.0.

Kernel Principal Component Analysis.

Sparse Principal Component Analysis.

Dimensionality reduction using truncated SVD.

Incremental Principal Component Analysis.

For `n_components == 'mle'`, this class uses the method from: Minka, T. P.. “Automatic choice of dimensionality for PCA”. In NIPS, pp. 598-604

Implements the probabilistic PCA model from: Tipping, M. E., and Bishop, C. M. (1999). "Probabilistic principal component analysis". Journal of the Royal Statistical Society: Series B (Statistical Methodology), 61(3), 611-622. via the `score` and `score_samples` methods.

For `svd_solver == 'arpack'`, refer to `scipy.sparse.linalg.svds`.

For `svd_solver == 'randomized'`, see: Halko, N., Martinsson, P. G., and Tropp, J. A. (2011). "Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions". SIAM review, 53(2), 217-288. and also Martinsson, P. G., Rokhlin, V., and Tygert, M. (2011). "A randomized algorithm for the decomposition of matrices". Applied and Computational Harmonic Analysis, 30(1), 47-68.

Fit the model with `X`.

Training data, where `n_samples` is the number of samples and `n_features` is the number of features.

Returns the instance itself.

Fit the model with `X` and apply the dimensionality reduction on `X`.

Training data, where `n_samples` is the number of samples and `n_features` is the number of features.

This method returns a Fortran-ordered array. To convert it to a C-ordered array, use `'np.ascontiguousarray'`.

Compute data covariance with the generative model.

$$\text{cov} = \text{components_T} * S^{**2} * \text{components_} + \text{sigma2} * \text{eye}(\text{n_features})$$
 where S^{**2} contains the explained variances, and `sigma2` contains the noise variances.

Estimated covariance of data.

Get output feature names for transformation.

The feature names out will be prefixed by the lowercased class name. For example, if the transformer outputs 3 features, then the feature names out are: ["class_name0", "class_name1", "class_name2"].

Only used to validate feature names with the names seen in fit.

Transformed feature names.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A MetadataRequest encapsulating routing information.

Get parameters for this estimator.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Compute data precision matrix with the generative model.

Equals the inverse of the covariance but computed with the matrix inversion lemma for efficiency.

Estimated precision of data.

Transform data back to its original space.

In other words, return an input X_{original} whose transform would be X .

New data, where n_{samples} is the number of samples and $n_{\text{components}}$ is the number of components.

Original data, where n_{samples} is the number of samples and n_{features} is the number of features.

If whitening is enabled, `inverse_transform` will compute the exact inverse operation, which includes reversing whitening.

Return the average log-likelihood of all samples.

See. "Pattern Recognition and Machine Learning" by C. Bishop, 12.2.1 p. 574 or <http://www.miketipping.com/papers/met-mppca.pdf>

Average log-likelihood of the samples under the current model.

Return the log-likelihood of each sample.

See. "Pattern Recognition and Machine Learning" by C. Bishop, 12.2.1 p. 574 or <http://www.miketipping.com/papers/met-mppca.pdf>

Log-likelihood of each sample under the current model.

Set output container.

See Introducing the `set_output` API for an example on how to use the API.

Configure output of transform and `fit_transform`.

"default": Default output format of a transformer

"pandas": DataFrame output

"polars": Polars output

None: Transform configuration is unchanged

Added in version 1.4: "polars" option was added.

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as Pipeline). The latter have parameters of the form <component>__<parameter> so that it's possible to update each component of a nested object.

Estimator parameters.

Apply dimensionality reduction to X.

X is projected on the first principal components previously extracted from a training set.

New data, where n_samples is the number of samples and n_features is the number of features.

Projection of X in the first principal components, where n_samples is the number of samples and n_components is the number of the components.

Image denoising using kernel PCA

Faces recognition example using eigenfaces and SVMs

A demo of K-Means clustering on the handwritten digits data

Column Transformer with Heterogeneous Data Sources

Selecting dimensionality reduction with Pipeline and GridSearchCV

Pipelining: chaining a PCA and a logistic regression

Concatenating multiple feature extraction methods

Principal Component Regression vs Partial Least Squares Regression

Faces dataset decompositions

Blind source separation using FastICA

FastICA on 2D point clouds

Principal Component Analysis (PCA) on Iris Dataset

Model selection with Probabilistic PCA and Factor Analysis (FA)

Comparison of LDA and PCA 2D projection of Iris dataset

Factor Analysis (with rotation) to visualize patterns

Multi-dimensional scaling

Explicit feature map approximation for RBF kernels

Multilabel classification

Balance model complexity and cross-validated score

Kernel Density Estimation

Dimensionality Reduction with Neighborhood Components Analysis

Importance of Feature Scaling

Release Highlights for scikit-learn 1.4

Release Highlights for scikit-learn 1.5

© Copyright 2007 - 2025, scikit-le